



PREMIER UNIVERSITY

Department of Computer Science & Engineering

Lab Report

Report No. : 09

Course Code : CSE 454

Course Title : Compiler Construction Laboratory

Report Title : Write a program to compute the FOLLOW set of a given context-free grammar.

Date of Submission : 10/21/2025

Submitted To:

Ms. Tanni Dhoom,
Assistant Professor,
Department of Computer Science and Engineering.

Submitted By:

Name	: MD Habibul Basher Saikat
ID	: 2104010202254
Program	: B.Sc. in CSE
Batch	: 40 th
Section	: C

<u>Remarks:</u>

Experiment No: 09

Experiment Name: Write a program to compute the FOLLOW set of a given context-free grammar.

Objectives:

- Learn what the FOLLOW set of a non-terminal means.
- Write a program that computes FOLLOW sets for all non-terminals.
- Use FIRST sets and grammar structure to compute FOLLOW correctly (including \$ end marker).
- Understand why removing left recursion / left factoring and computing FIRST first is important.

Algorithm:

Prerequisites:

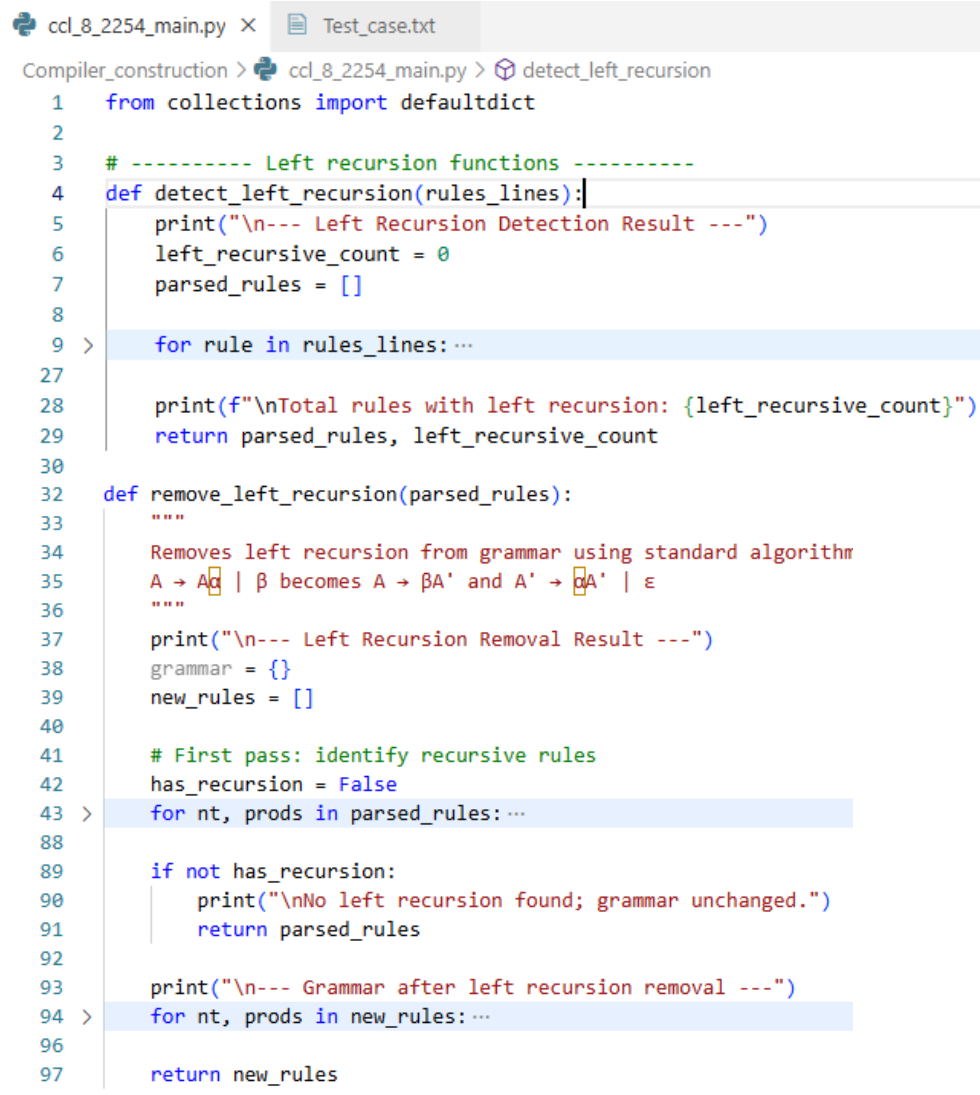
1. **Remove left recursion:** Remove direct and indirect left recursion from the grammar first. Left recursion can confuse some parser-building steps and may introduce cycles that are harder to reason about.
2. **Apply left factoring:** If two productions share a common prefix, factor them so choices are explicit. This helps deterministic parsing and clearer FOLLOW set behavior later.
3. **Compute FIRST sets:** FOLLOW computation depends on FIRST sets for sequences of symbols. Compute FIRST for all terminals and non-terminals before starting FOLLOW.
4. **Decide tokenization mode:** Use tokenized productions (lists of symbols) so multi-character non-terminals are treated correctly.

Step-by-step FOLLOW algorithm (simple):

1. **Input:** A tokenized grammar (productions as lists of symbols), the set of non-terminals N , the set of terminals T , and FIRST sets already computed. Decide the start symbol S (usually the left-hand side of the first production).
2. **Initialize:**
 - a. For each non-terminal A in N , set $\text{FOLLOW}(A) = \{ \}$ (empty set).
 - b. Add the end-marker $\$$ to $\text{FOLLOW}(S)$ where S is the start symbol (meaning input can end after S).
3. **Repeat until no change:**
 - a. For every production $A \rightarrow X_1 X_2 \dots X_k$:
 - i. For every symbol X_i in the production where X_i is a non-terminal:
 1. Let $\text{beta} = X_{i+1} \dots X_k$ (the sequence that follows X_i in this production).
 2. Compute $\text{FIRST}(\text{beta})$ (the FIRST set of the sequence beta):
 - a. If beta is empty, then $\text{FIRST}(\text{beta}) = \{ \epsilon \}$.
 - b. Otherwise, $\text{FIRST}(\text{beta})$ is the union of $\text{FIRST}(X_{i+1})$ minus ϵ , and if X_{i+1} can produce ϵ then include $\text{FIRST}(X_{i+2})$, etc.
 3. **Add** all terminals in $\text{FIRST}(\text{beta})$ except ϵ to $\text{FOLLOW}(X_i)$.
 4. **If** $\epsilon \in \text{FIRST}(\text{beta})$ (i.e., the entire beta can produce ϵ) **or** beta is empty:

- a. Add all symbols in FOLLOW(A) to FOLLOW(Xi). (Because if beta can vanish, whatever can follow A can also follow Xi.)
 - b. Continue scanning all productions and keep updating FOLLOW sets until a whole pass makes no changes.
4. **Finish:**
 - a. When no FOLLOW set changes in a complete pass, the algorithm finishes.
 - b. Output FOLLOW(A) for each non-terminal A.

Code Snippet:



```

ccl_8_2254_main.py X Test_case.txt
Compiler_construction > ccl_8_2254_main.py > detect_left_recursion
1  from collections import defaultdict
2
3  # ----- Left recursion functions -----
4  def detect_left_recursion(rules_lines):
5      print("\n--- Left Recursion Detection Result ---")
6      left_recursive_count = 0
7      parsed_rules = []
8
9  >   for rule in rules_lines: ...
27
28   print(f"\nTotal rules with left recursion: {left_recursive_count}")
29   return parsed_rules, left_recursive_count
30
31  def remove_left_recursion(parsed_rules):
32      """
33      Removes left recursion from grammar using standard algorithm
34      A → Aβ | β becomes A → βA' and A' → βA' | ε
35      """
36
37      print("\n--- Left Recursion Removal Result ---")
38      grammar = {}
39      new_rules = []
40
41      # First pass: identify recursive rules
42      has_recursion = False
43  >   for nt, prods in parsed_rules: ...
88
89   if not has_recursion:
90       print("\nNo left recursion found; grammar unchanged.")
91       return parsed_rules
92
93   print("\n--- Grammar after left recursion removal ---")
94  >   for nt, prods in new_rules: ...
96
97   return new_rules
--

```

Figure 1: Left Recursion Code Snippet

```

100 # ----- Left factoring functions -----
101 EPSILON = 'ε'
102
103 > class _TrieNode: ...
107
108 > def _choose_char_mode(productions): ...
113
114 > def _tokenize_char_mode_with_nts(prod, nonterminals): ...
137
138 > def _tokenize(prod, char_mode, nonterminals=None): ...
149
150 > def _build_trie_with_mode(productions, char_mode, nonterminals=None): ...
162
163 > def _collect_maximal_prefixes(root): ...

```

```

176 def detection_left_factoring(parsed_rules, verbose=True):
177     factoring_map = {}
178     total_groups = 0
179
180 > for nt, prods in parsed_rules: ...
205 > if verbose:
206     print(f"\nTotal left factoring groups in grammar: {total_groups}")
207     return factoring_map, total_groups
208
209 > def _unique_new_nt(base, existing_set): ...
217
218 def removal_left_factoring(parsed_rules):
219     print("\n--- Left Factoring Removal Result ---")
220     grammar = {nt: list(prods) for nt, prods in parsed_rules}
221     existing_nts = set(grammar.keys())
222
223     iteration = 0
224 > while True: ...
286
287     print("\n--- Grammar after left factoring removal ---")
288     final_rules = []
289 > for nt, prods in grammar.items():
290     print(f"{nt} → {' | '.join(prods)}")
291     final_rules.append((nt, prods))
292     return final_rules
---
```

Figure 2: Left Factoring Code Snippet

```

295 # ----- FIRST sets -----
296 def build_tokenized_grammar(parsed_rules):
297     nonterminals = [nt for nt, _ in parsed_rules]
298     char_mode_map = {}
299     for nt, prods in parsed_rules:
300         char_mode_map[nt] = _choose_char_mode(prods)
301
302     grammar_tokens = {}
303     for nt, prods in parsed_rules:
304         cm = char_mode_map.get(nt, True)
305         tokenized_prods = []
306         current_nts = set([x for x, _ in parsed_rules])
307         for p in prods:
308             if cm:
309                 tokens = _tokenize(p, True, current_nts)
310             else:
311                 tokens = _tokenize(p, False, None)
312             tokenized_prods.append(tokens)
313         grammar_tokens[nt] = tokenized_prods
314
315     terminals = set()
316     for nt, prods in grammar_tokens.items():
317         for prod in prods:
318             for tok in prod:
319                 if tok == EPSILON:
320                     continue
321                 if tok not in grammar_tokens.keys():
322                     terminals.add(tok)
323     return grammar_tokens, char_mode_map, set(grammar_tokens.keys()), terminals
324
```

```

325 def compute_first_sets(grammar_tokens):
326     FIRST = defaultdict(set)
327
328     nonterminals = set(grammar_tokens.keys())
329     symbols = set()
330     for nt, prods in grammar_tokens.items():
331         for prod in prods:
332             for tok in prod:
333                 symbols.add(tok)
334
335     terminals = set([s for s in symbols if s not in nonterminals and s != EPSILON])
336     for t in terminals:
337         FIRST[t].add(t)
338
339     for nt in nonterminals:
340         FIRST[nt] = set()
341
342     changed = True
343 > while changed: ...
368     return FIRST
369
370 def pretty_print_first_sets(FIRST):
371     print("\n--- FIRST sets ---")
372     for sym in sorted(FIRST.keys()):
373         if sym.isprintable():
374             items = ', '.join(sorted(FIRST[sym]))
375             print(f"FIRST({sym}) = {{ {items} }}")
376
370 < def pretty_print_first_sets(FIRST):
371     print("\n--- FIRST sets ---")
372 < for sym in sorted(FIRST.keys()):
373 <     if sym.isprintable():
374         items = ', '.join(sorted(FIRST[sym]))
375         print(f"FIRST({sym}) = {{ {items} }}")
376

```

Figure 3: First Set Code Snippet

```

378 # ----- FOLLOW sets -----
379 def first_of_sequence(seq, FIRST):
380     if not seq:
381         return {EPSILON}
382     result = set()
383     for symbol in seq:
384         if symbol not in FIRST:
385             result.add(symbol)
386             return result
387         result |= (FIRST[symbol] - {EPSILON})
388         if EPSILON in FIRST[symbol]:
389             continue
390         else:
391             return result
392     result.add(EPSILON)
393     return result
394

```

```

395 def compute_follow_sets(grammar_tokens, FIRST, start_symbol):
396     nonterminals = list(grammar_tokens.keys())
397     FOLLOW = {nt: set() for nt in nonterminals}
398     FOLLOW[start_symbol].add('$')
399
400     changed = True
401     while changed:
402         changed = False
403         for A, prods in grammar_tokens.items():
404             for prod in prods:
405                 for i, B in enumerate(prod):
406                     if B not in grammar_tokens:
407                         continue
408                     beta = prod[i+1:]
409                     first_beta = first_of_sequence(beta, FIRST)
410                     to_add = set(first_beta) - {EPSILON}
411                     if to_add - FOLLOW[B]:
412                         FOLLOW[B].update(to_add)
413                         changed = True
414                     if EPSILON in first_beta or not beta:
415                         if FOLLOW[A] - FOLLOW[B]:
416                             FOLLOW[B].update(FOLLOW[A])
417                             changed = True
418     return FOLLOW

420 def pretty_print_follow_sets(FOLLOW):
421     print("\n--- FOLLOW sets ---")
422     for sym in sorted(FOLLOW.keys()):
423         if sym.isprintable():
424             items = ', '.join(sorted(FOLLOW[sym]))
425             print(f"FOLLOW({sym}) = {{ {items} }}")
426

```

Figure 4: Follow Set Code Snippet

```

504 # -----
505 # MAIN PROGRAM
506 # -----
507 if __name__ == "__main__":
508     n = int(input("Enter number of rules: "))
509     rules = []
510     for _ in range(n):
511         rules.append(input().strip())
512
513     print("\n" + "=" * 70)
514     print("STEP 1: LEFT RECURSION DETECTION")
515     print("=" * 70)
516
517     # 1) Left recursion detection
518     parsed_rules, left_recursive_count = detect_left_recursion(rules)
519
520     print("\n" + "=" * 70)
521     print("STEP 2: LEFT RECURSION REMOVAL")
522     print("=" * 70)
523
524     # 2) Left recursion removal
525     rules_after_lr = remove_left_recursion(parsed_rules)
526
527     print("\n" + "=" * 70)
528     print("STEP 3: LEFT FACTORING DETECTION")
529     print("=" * 70)
530
531     # 3) Left factoring detection
532     factoring_map, total_fact_groups = detection_left_factoring(rules_after_lr, verbose=True)
533
534     print("\n" + "=" * 70)
535     print("STEP 4: LEFT FACTORING REMOVAL")
536     print("=" * 70)
537
538     # 4) Left factoring removal
539     if total_fact_groups > 0:
540         final_grammar = removal_left_factoring(rules_after_lr)
541     else:
542         print("\n--- Left Factoring Removal Result ---")
543         print("\nNo left factoring groups found; grammar unchanged after factoring pass.")
544         final_grammar = rules_after_lr
545
546     print("\n" + "=" * 70)
547     print("STEP 5: BUILD TOKENIZED GRAMMAR & COMPUTE FIRST SETS")
548     print("=" * 70)
549
550     # 5) Build tokenized grammar
551     grammar_tokens, char_mode_map, nonterminals, terminals = build_tokenized_grammar(final_grammar)
552
553     # Choose start symbol
554     if final_grammar:
555         start_symbol = final_grammar[0][0]
556     else:
557         start_symbol = None
558
559     print(f"\nStart Symbol: {start_symbol}")
560     print(f"Non-terminals: {sorted(nonterminals)}")
561     print(f"Terminals: {sorted(terminals)}")
562
563     # 6) Compute FIRST sets
564     FIRST = compute_first_sets(grammar_tokens)
565     pretty_print_first_sets(FIRST)
566
567     print("\n" + "=" * 70)
568     print("STEP 6: COMPUTE FOLLOW SETS")
569     print("=" * 70)

```

```

571         # 7) Compute FOLLOW sets
572         FOLLOW = compute_follow_sets(grammar_tokens, FIRST, start_symbol)
573         pretty_print_follow_sets(FOLLOW)
574
575         print("\n" + "=" * 70)
576         print("STEP 7: CONSTRUCT LL(1) PARSING TABLE")
577         print("=" * 70)
578     ---

```

Output Snippet:

PS C:\Users\Habib\Desktop> & "C:\Program Files\Python314\python.exe" c:/Users/Habib/Desktop/Compiler_construction/ccl_8_2254_main.py

Enter number of rules: 3

E->E+T|T

T->T*F|F

F->(E)|id

```

=====
STEP 1: LEFT RECURSION DETECTION
=====

```

--- Left Recursion Detection Result ---

E has 1 left recursive production(s):

E → E+T

T has 1 left recursive production(s):

T → T*F

F has no left recursion.

Total rules with left recursion: 2

```

-----
--- Left Recursion Removal Result ---

```

Removing left recursion from E:

Left recursive productions: E+T

Non-recursive productions: T

Introduced E':

New E → TE'

E' → +TE', ε

Removing left recursion from T:

Left recursive productions: T*F

Non-recursive productions: F

Introduced T':

New T → FT'

T' → *FT', ε

--- Grammar after left recursion removal ---

E → TE'

E' → +TE' | ε

T → FT'

T' → *FT' | ε

F → (E) | id

=====

STEP 3: LEFT FACTORING DETECTION

=====

E has no left factoring.
E' has no left factoring.
T has no left factoring.
T' has no left factoring.
F has no left factoring.

Total left factoring groups in grammar: 0

=====

STEP 4: LEFT FACTORING REMOVAL

=====

--- Left Factoring Removal Result ---

No left factoring groups found; grammar unchanged after factoring pass.

=====

STEP 5: BUILD TOKENIZED GRAMMAR & COMPUTE FIRST SETS

=====

Start Symbol: E
Non-terminals: ['E', "E'", 'F', 'T', "T'"]
Terminals: ['(', ')', '*', '+', 'd', 'i']

--- FIRST sets ---

FIRST(() = { (}
FIRST() = {) }
FIRST(*) = { * }
FIRST(+) = { + }
FIRST(E) = { (, i }
FIRST(E') = { +, ε }
FIRST(F) = { (, i }
FIRST(T) = { (, i }
FIRST(T') = { *, ε }
FIRST(d) = { d }
FIRST(i) = { i }

=====

STEP 6: COMPUTE FOLLOW SETS

=====

--- FOLLOW sets ---

FOLLOW(E) = { \$,) }
FOLLOW(E') = { \$,) }
FOLLOW(F) = { \$,), *, + }
FOLLOW(T) = { \$,), + }
FOLLOW(T') = { \$,), + }

Discussion: The FOLLOW set tells us which terminals can appear immediately after a non-terminal in any valid string derived from the grammar. To compute FOLLOW sets correctly, we must already know the FIRST sets, and the grammar should be free from left recursion and left factoring. In this experiment, I first prepared the grammar by removing these issues, then calculated FIRST sets to support FOLLOW computation. The algorithm begins by assigning \$ to the FOLLOW of the start symbol. Then, for each production, whenever a non-terminal B appears before another symbol β , I added all terminals from $\text{FIRST}(\beta)$ (except ϵ) to $\text{FOLLOW}(B)$. If β can produce ϵ or if B is at the end of a production, I added everything from $\text{FOLLOW}(A)$ (where $A \rightarrow \alpha B$) to $\text{FOLLOW}(B)$. I repeated these steps until no FOLLOW set changed further. This experiment helped me clearly understand how FOLLOW sets work in building parsing tables. FOLLOW sets ensure that ϵ -productions and end-of-input conditions are handled correctly in predictive parsing.